


```
# Run this first, before anything else
from google.colab import drive
drive.mount('/content/drive')

import os

BASE = '/content/drive/MyDrive/CLTT_Soccer'
FRAMES_DRIVE = f'{BASE}/frames'
CKPT_DIR = f'{BASE}/checkpoints'
UCF_DIR = f'{BASE}/ucf101'

for d in [BASE, FRAMES_DRIVE, CKPT_DIR, UCF_DIR]:
    os.makedirs(d, exist_ok=True)

print("Drive mounted. Directories created.")
```

```
Mounted at /content/drive
Drive mounted. Directories created.
```

```
!pip install -q yt-dlp decord av
```

```
_____ 182.3/182.3 kB 14.7 MB/s et
_____ 3.3/3.3 MB 128.2 MB/s eta
_____ 13.6/13.6 MB 144.7 MB/s et
_____ 36.3/36.3 MB 77.2 MB/s eta
```

```
VIDEO_URL = "https://www.youtube.com/watch?v=gTmR5PMIIjs" # ← only
VIDEO_PATH = f'{BASE}/soccer_asmr.mp4'

if not os.path.exists(VIDEO_PATH):
    !yt-dlp -f "bestvideo[height<=720][ext=mp4]+bestaudio[ext=m4a]/be:
else:
    print("Video already on Drive, skipping download.")
```

```
Video already on Drive, skipping download.
```

```
LOCAL_VIDEO = '/content/soccer_asmr.mp4'

if not os.path.exists(LOCAL_VIDEO):
    print("Copying video to local SSD...")
    !cp "{VIDEO_PATH}" "{LOCAL_VIDEO}"
    print(f"Done: {os.path.getsize(LOCAL_VIDEO)/1e9:.2f} GB")

print("Video ready.")
```

```
Copying video to local SSD...
Done: 2.15 GB
Video ready.
```

```
import os

os.makedirs('/content/soccer_frames', exist_ok=True)

# Video is already local – ffmpeg reads it sequentially (very fast)
!ffmpeg -i /content/soccer_asmr.mp4 -vf "fps=5,scale=128:128" -q:v 3

n = len([f for f in os.listdir('/content/soccer_frames') if f.endswith('jpg')])
print(f"Extracted {n} frames")
```

[swscaler @ 0x58ed6fcaaa00] deprecated pixel format used, make sure you
 Extracted 55075 frames

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.models.video as video_models
from PIL import Image
import random, numpy as np
from tqdm import tqdm

# — Kinetics-400 normalization stats (standard for video models) —
MEAN = [0.43216, 0.39467, 0.37645]
STD = [0.22803, 0.22145, 0.21699]

# —————
from PIL import Image
import os, random, torch
from torch.utils.data import Dataset

class CLTTDataset(Dataset):
    def __init__(self, frames_dir, clip_len=8, fps=5,
                 pos_window_sec=5, transform=None):
        self.frames_dir = frames_dir
        self.clip_len = clip_len
        self.pos_window = int(pos_window_sec * fps)
        self.transform = transform
        self.frames = sorted(f for f in os.listdir(frames_dir)
                             if f.endswith('.jpg'))

        self.N = len(self.frames)
        print(f"{self.N} frames | {self.N/fps/3600:.2f} hrs | "
              f"pos_window={pos_window_sec}s")

    def _load_clip(self, start):
        imgs = []
        for i in range(self.clip_len):
            idx = min(start + i, self.N - 1)
            img = Image.open(os.path.join(self.frames_dir, self.frames[idx]))
            if self.transform is not None:
                img = self.transform(img)
```

```

        img = image.open(r'{self.trares_dir}/{self.trares[idx]}')
        imgs.append(self.transform(img) if self.transform else img)
    return torch.stack(imgs, dim=1)    # C, T, H, W

def __len__(self):
    return max(0, self.N - self.clip_len - self.pos_window - 1)

def __getitem__(self, idx):
    clip_a = self._load_clip(idx)
    offset = random.randint(1, self.pos_window)
    clip_b = self._load_clip(min(idx + offset, self.N - self.clip_len))
    return clip_a, clip_b

# -----
class CLTTModel(nn.Module):
    """R(2+1)D-18 encoder + MLP projection head."""
    def __init__(self, proj_dim=128):
        super().__init__()
        backbone = video_models.r2plus1d_18(weights=None)

        # Strip the classification FC; keep everything else
        self.stem = backbone.stem
        self.layer1 = backbone.layer1
        self.layer2 = backbone.layer2
        self.layer3 = backbone.layer3
        self.layer4 = backbone.layer4
        self.avgpool = backbone.avgpool    # → (B, 512, 1, 1, 1)

        # Projection head used only during pretraining
        self.projector = nn.Sequential(
            nn.Linear(512, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Linear(512, proj_dim),
        )

    def encode(self, x):
        """512-dim backbone features – used for downstream tasks."""
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)
        return x.flatten(1)    # (B, 512)

    def forward(self, x):
        """Normalized projected features – used during pretraining."""
        z = self.projector(self.encode(x))
        return F.normalize(z, dim=-1)    # (B, proj_dim)

```

```
# -----
class NTXentLoss(nn.Module):
    """NT-Xent (Normalized Temperature-scaled Cross Entropy)."""
    def __init__(self, temperature=0.07):
        super().__init__()
        self.T = temperature

    def forward(self, z_a, z_b):
        B = z_a.size(0)
        z = torch.cat([z_a, z_b], dim=0) # (2B,
        sim = torch.mm(z, z.T) / self.T # (2B,

        # Mask out self-similarity on the diagonal
        mask = torch.eye(2 * B, dtype=torch.bool, device=z.device)
        sim = sim.masked_fill(mask, float('-inf'))

        # Positive i pairs with i+B, and i+B pairs with i
        labels = torch.cat([
            torch.arange(B, 2 * B, device=z.device),
            torch.arange(0, B, device=z.device),
        ])
        return F.cross_entropy(sim, labels)

print("Classes defined.")
```

Classes defined.

```
import torch.optim as optim
from torch.cuda.amp import autocast, GradScaler

def run_pretraining(frames_dir, ckpt_dir, epochs=2, batch_size=32, lr:
    device = torch.device('cuda' if torch.cuda.is_available() else 'c
    print(f"Device: {device}")

    train_transform = T.Compose([
        T.RandomCrop(112),
        T.RandomHorizontalFlip(),
        T.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, h
        T.ToTensor(),
        T.Normalize(MEAN, STD),
    ])

    dataset = CLTTDataset(frames_dir, clip_len=8, fps=5,
                           pos_window_sec=5, transform=train_transform

    loader = DataLoader(dataset, batch_size=32, shuffle=True,
                        num_workers=4, pin_memory=True, drop_last=True

    model = CLTTModel(proj_dim=128).to(device)
    criterion = NTXentLoss(temperature=0.07)
```

```

optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=epochs * len(loader))
scaler = torch.amp.GradScaler('cuda')

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    pbar = tqdm(loader, desc=f'Epoch {epoch+1}/{epochs}')

    for clip_a, clip_b in pbar:
        clip_a, clip_b = clip_a.to(device), clip_b.to(device)

        optimizer.zero_grad()
        with autocast():
            z_a = model(clip_a)
            z_b = model(clip_b)
            loss = criterion(z_a, z_b)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()

        running_loss += loss.item()
        pbar.set_postfix(loss=f'{loss.item():.4f}')

    avg = running_loss / len(loader)
    print(f'Epoch {epoch+1} - avg loss: {avg:.4f}')

    ckpt = f'{ckpt_dir}/cltt_epoch{epoch+1}.pt'
    torch.save({'epoch': epoch+1,
                'model_state_dict': model.state_dict(),
                'loss': avg}, ckpt)
    print(f'Saved → {ckpt}')

return model

print("Pretraining function ready.")

```

Pretraining function ready.

```
LOCAL_FRAMES = '/content/soccer_frames'
```

```

pretrained_model = run_pretraining(
    frames_dir = LOCAL_FRAMES,
    ckpt_dir    = CKPT_DIR,
    epochs      = 2,
    batch_size  = 32,
    lr          = 3e-4,

```

```
)
```

```
Device: cuda
55075 frames | 3.06 hrs | pos_window=5s
Epoch 1/2: 0%|██████████| 0/1720 [00:00<?, ?it/s]/tmp/ipykernel_796/
  with autocast():
/tmp/ipykernel_796/2739935460.py:46: UserWarning: Detected call of `lr
scheduler.step()`
Epoch 1/2: 100%|██████████| 1720/1720 [09:35<00:00, 2.99it/s, loss=0.
Epoch 1 – avg loss: 0.8642
Saved → /content/drive/MyDrive/CLTT_Soccer/checkpoints/cltt_epoch1.pt
Epoch 2/2: 100%|██████████| 1720/1720 [09:35<00:00, 2.99it/s, loss=0.
Epoch 2 – avg loss: 0.3287
Saved → /content/drive/MyDrive/CLTT_Soccer/checkpoints/cltt_epoch2.pt
```

```
import shutil

ucf101_dir = f'{UCF_DIR}/UCF-101'

if os.path.exists(ucf101_dir):
    n = sum(len(os.listdir(f'{ucf101_dir}/{c}')) for c in os.listdir(ucf101_dir))
    print(f"UCF-101 ready – {n} videos across {len(os.listdir(ucf101_dir))} classes")
else:
    split_dirs_exist = any(os.path.exists(f'{UCF_DIR}/{s}') for s in ['train', 'test', 'val'])
    if not split_dirs_exist:
        zip_path = f'{UCF_DIR}/archive.zip'
        if not os.path.exists(zip_path):
            raise FileNotFoundError(f"archive.zip not found at {zip_path}")
        print("Extracting UCF101 from Drive archive.zip...")
        !unzip -oq "{zip_path}" -d "{UCF_DIR}/"

    print("Merging train/test/val into UCF-101/ (one-time, may take a few minutes)")
    os.makedirs(ucf101_dir, exist_ok=True)
    for split in ['train', 'test', 'val']:
        split_dir = f'{UCF_DIR}/{split}'
        if not os.path.exists(split_dir):
            continue
        for cls in os.listdir(split_dir):
            cls_dst = f'{ucf101_dir}/{cls}'
            os.makedirs(cls_dst, exist_ok=True)
            for video in os.listdir(f'{split_dir}/{cls}'):
                src = f'{split_dir}/{cls}/{video}'
                dst = f'{cls_dst}/{video}'
                if not os.path.exists(dst):
                    shutil.move(src, dst)

    n = sum(len(os.listdir(f'{ucf101_dir}/{c}')) for c in os.listdir(ucf101_dir))
    print(f"Done – {n} videos across {len(os.listdir(ucf101_dir))} classes")
```

```
Merging train/test/val into UCF-101/ (one-time, may take a few minutes)
Done – 13451 videos across 101 classes.
```

```

splits_dir = f'{UCF_DIR}/ucfTrainTestlist'
n_splits = len(os.listdir(splits_dir)) if os.path.exists(splits_dir) else 0

if n_splits < 6:
    drive_zip = f'{UCF_DIR}/splits.zip'
    if os.path.exists(drive_zip):
        print("Found splits.zip on Drive – extracting...")
        !unzip -o "{drive_zip}" -d /tmp/ucf_splits_dir/
    else:
        print("Downloading UCF101 train/test splits from UCF website.")
        !wget -c "https://www.crcv.ucf.edu/data/UCF101/UCF101TrainTestSplitsDataRelease/ucfTrainTestlist.zip"
        -O /tmp/ucf_splits.zip
        !unzip -o /tmp/ucf_splits.zip -d /tmp/ucf_splits_dir/
        !cp -rf /tmp/ucf_splits_dir/ucfTrainTestlist "{UCF_DIR}/"
        print(f"Splits ready ({len(os.listdir(splits_dir))} files).")
    else:
        print(f"Splits already present ({n_splits} files).")

```

Splits already present (7 files).

```

from torchvision.datasets import UCF101

device = torch.device('cuda')

def make_ucf_transform():
    return T.Compose([
        T.Resize(128),
        T.CenterCrop(112),
        T.ConvertImageDtype(torch.float32),
        T.Normalize(MEAN, STD),
    ])

def ucf_collate(batch):
    # audio has variable length across clips – discard it
    videos = torch.stack([item[0] for item in batch])
    labels = torch.tensor([item[2] for item in batch])
    return videos, None, labels

def load_ucf101(ucf_dir, split='train', fold=1):
    return UCF101(
        root = f'{ucf_dir}/UCF-101',
        annotation_path = f'{ucf_dir}/ucfTrainTestlist',
        frames_per_clip = 16,
        step_between_clips = 8,
        fold = fold,
        train = (split == 'train'),
        transform = make_ucf_transform(),
        output_format = 'TCHW',
    )

```



```

@torch.no_grad()
def extract_features(model, loader, device):
    model.eval()
    features, labels = [], []
    for videos, _, lbls in tqdm(loader, desc='Extracting'):
        x = videos.permute(0, 2, 1, 3, 4).to(device)
        h = model.encode(x).cpu()
        features.append(h)
        labels.append(lbls)
    return torch.cat(features), torch.cat(labels)

train_ds = load_ucf101(UCF_DIR, 'train')
test_ds = load_ucf101(UCF_DIR, 'test')

train_loader = DataLoader(train_ds, batch_size=16, num_workers=4, colla
test_loader = DataLoader(test_ds, batch_size=16, num_workers=4, colla

print("Extracting train features...")
train_feats, train_labels = extract_features(pretrained_model, train_lo
print("Extracting test features...")
test_feats, test_labels = extract_features(pretrained_model, test_loa

print(f"Train: {train_feats.shape}, Test: {test_feats.shape}")
torch.save({'feats': train_feats, 'labels': train_labels}, f'{CKPT_DIR}
torch.save({'feats': test_feats, 'labels': test_labels}, f'{CKPT_DIR}
print("Features saved.")

```

```

0%|          | 0/841 [00:00<?, ?it/s]/usr/local/lib/python3.12/dist-
warnings.warn(
100%|██████████| 841/841 [06:16<00:00, 2.24it/s]
0%|          | 0/841 [00:00<?, ?it/s]/usr/local/lib/python3.12/dist-
warnings.warn(
100%|██████████| 841/841 [06:14<00:00, 2.25it/s]
Extracting train features...
Extracting: 0%|          | 0/13099 [00:00<?, ?it/s]/usr/local/lib/py
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecat
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecat
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecat
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
Extracting: 100%|██████████| 13099/13099 [1:39:44<00:00, 2.19it/s]
Extracting test features...

```

```

Extracting: 0%|          | 0/5109 [00:00<?, ?it/s]/usr/local/lib/pyt
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecat
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecat
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecat
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
/usr/local/lib/python3.12/dist-packages/torchvision/io/video.py:199: U
warnings.warn("The pts_unit 'pts' gives wrong results. Please use pt
Extracting: 100%|██████████| 5109/5109 [39:07<00:00, 2.18it/s]
Train: torch.Size([209582, 512]), Test: torch.Size([81743, 512])
Features saved.

```

```

from torch.utils.data import TensorDataset

# Load from Drive if needed
train_data = torch.load(f'{CKPT_DIR}/train_feats.pt')
test_data = torch.load(f'{CKPT_DIR}/test_feats.pt')
train_feats, train_labels = train_data['feats'], train_data['labels']
test_feats, test_labels = test_data['feats'], test_data['labels']

NUM_CLASSES = 101

# — Linear probe: frozen features → trainable linear head —————
probe = nn.Linear(512, NUM_CLASSES).to(device)
opt = optim.AdamW(probe.parameters(), lr=1e-3, weight_decay=1e-4)

probe_train = DataLoader(TensorDataset(train_feats, train_labels),
                          batch_size=256, shuffle=True)
probe_test = DataLoader(TensorDataset(test_feats, test_labels),
                        batch_size=256)

for epoch in range(20):
    probe.train()
    for x, y in probe_train:
        x, y = x.to(device), y.to(device)
        loss = F.cross_entropy(probe(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()

    # Eval every 5 epochs
    if (epoch + 1) % 5 == 0:
        probe.eval()
        correct = total = 0

```

```
correct = total = 0
with torch.no_grad():
    for x, y in probe_test:
        x, y = x.to(device), y.to(device)
        preds = probe(x).argmax(dim=1)
        correct += (preds == y).sum().item()
        total += y.size(0)
print(f"Epoch {epoch+1:2d} - Test Acc: {100*correct/total:.2f}%")
```

```
Epoch  5 - Test Acc: 20.97%
Epoch 10 - Test Acc: 20.72%
Epoch 15 - Test Acc: 21.59%
Epoch 20 - Test Acc: 22.23%
```